1. Employees in an organization need to be grouped for a tournament based on their ages. Sort the ages using Merge sort and find the time required to perform the sorting.

Description of Merge Sort:

Merge Sort is a divide-and-conquer algorithm that divides the input array into two halves, recursively sorts them, and then merges the sorted halves to produce the sorted result. The key idea of merge sort is to break down the array into smaller arrays, sort those, and combine them. It has a time complexity of O(n log n), where n is the number of elements in the array.

C Program to Sort Ages Using Merge Sort:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
// Function to merge two halves of the array
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int leftArr[n1], rightArr[n2];

    // Copy data to temporary arrays
    for (int i = 0; i < n1; i++)
        leftArr[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        rightArr[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    // Merge the temporary arrays back into the original array
    while (i < n1 && j < n2) {
        if (leftArr[i] <= rightArr[j]) {
            arr[k] = leftArr[i];
            i++;
        } else {
            arr[k] = rightArr[j];
            j++;
        }
        k++;
    }

    // Copy the remaining elements of leftArr[], if any
    while (i < n1) {
        arr[k] = leftArr[i];
        i++;
        k++;
    }
```

```
    // Copy the remaining elements of rightArr[], if any
    while (j < n2) {
        arr[k] = rightArr[j];
        j++;
        k++;
    }
}
// Function to implement merge sort
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        // Recursively sort the two halves
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        // Merge the sorted halves
        merge(arr, left, mid, right);
    }
}
int main() {
    int n;

    // Input number of employees (size of the array)
    printf("Enter the number of employees: ");
    scanf("%d", &n);

    int ages[n];

    // Input ages of employees
    printf("Enter the ages of the employees:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &ages[i]);
    }

    // Start the clock to calculate time
    clock_t start_time = clock();

    // Perform merge sort on the ages array
    mergeSort(ages, 0, n - 1);

    // End the clock after sorting
    clock_t end_time = clock();

    // Calculate the time taken for sorting
    double time_taken = ((double)(end_time - start_time)) / CLOCKS_PER_SEC;
    // Output the sorted ages
    printf("Sorted ages of employees:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", ages[i]);
```

```
    }
    printf("\n");

    // Output the time taken for sorting
    printf("Time taken for sorting: %.6f seconds\n", time_taken);

    return 0;
}
```

Explanation of the Code:

1. **Merge Function**: This function takes an array and the indices of two subarrays, merges them in sorted order.
2. **Merge Sort Function**: It recursively splits the array into two halves, sorts each half, and then merges them back together in sorted order.
3. **Main Function**: It reads the number of employees and their ages, calls the mergeSort function to sort the ages, and then calculates and prints the time taken to perform the sorting.

Example Input/Output:
Enter the number of employees: 5
Enter the ages of the employees:35 25 50 40 30
Sorted ages of employees:25 30 35 40 50
Time taken for sorting: 0.000015 seconds
Time Complexity:

- The time complexity of Merge Sort is **O(n log n)** for both the average and worst-case scenarios.

2. Students in a department need to be selected for a high jump competition based on their height (integer values only). Sort the heights of students using Quick sort and find the time required for the sorting.

Description of Quick Sort:
Quick Sort is a divide-and-conquer sorting algorithm that works by selecting a 'pivot' element from the array and partitioning the other elements into two sub-arrays according to whether they are less than or greater than the pivot. The sub-arrays are then recursively sorted.
The basic steps for Quick Sort are:

1. **Partitioning**: Pick a pivot element from the array, and rearrange the elements so that all elements smaller than the pivot come before it, and all elements greater come after it.
2. **Recursion**: Recursively apply the same logic to the left and right sub-arrays.

The time complexity of Quick Sort is **O(n log n)** on average, but in the worst case, it can be **O(n²)**, depending on the pivot selection.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
// Function to partition the array into two sub-arrays
   int partition(int arr[], int low, int high)
   {
   int pivot = arr[high];  // Choose the last element as pivot
   int i = (low - 1);  // Index of the smaller element

   // Rearrange elements in the array
   for (int j = low; j < high; j++) {
      if (arr[j] <= pivot) {
         i++;
         // Swap arr[i] and arr[j]
         int temp = arr[i];
         arr[i] = arr[j];
         arr[j] = temp;
      }
   }

   // Swap arr[i + 1] and arr[high] (pivot)
   int temp = arr[i + 1];
   arr[i + 1] = arr[high];
   arr[high] = temp;

   return (i + 1);  // Return the pivot index
}
// Function to implement Quick Sortvoid quickSort(int arr[], int low, int high) {
   if (low < high) {
      // Partition the array into two halves
      int pi = partition(arr, low, high);

      // Recursively sort the left and right sub-arrays
      quickSort(arr, low, pi - 1);
      quickSort(arr, pi + 1, high);
   }
}
int main()
 {
   int n;
```

```
    // Input the number of students
    printf("Enter the number of students: ");
    scanf("%d", &n);

    int heights[n];

    // Input the heights of the students
    printf("Enter the heights of the students:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &heights[i]);
    }

    // Start the clock to measure time taken for sorting
    clock_t start_time = clock();

    // Perform quick sort on the heights array
    quickSort(heights, 0, n - 1);

    // End the clock after sorting
    clock_t end_time = clock();

    // Calculate the time taken for sorting
    double time_taken = ((double)(end_time - start_time)) / CLOCKS_PER_SEC;

    // Output the sorted heights
    printf("Sorted heights of students:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", heights[i]);
    }
    printf("\n");

    // Output the time taken for sorting
    printf("Time taken for sorting: %.6f seconds\n", time_taken);

    return 0;
}
```

Explanation of the Code:

1. **Partition Function**: The partition function selects a pivot (in this case, the last element) and rearranges the array so that elements less than the pivot are placed before it, and elements greater than the pivot are placed after it.
2. **Quick Sort Function**: The quickSort function recursively divides the array into smaller sub-arrays and sorts them by partitioning each array around a pivot.
3. **Main Function**:

    1. It takes the number of students and their heights as input.
    2. Calls the quickSort function to sort the heights.
    3. Measures and prints the time taken to sort the array using the clock function.

Example Input/Output:
Input:
Enter the number of students: 6
Enter the heights of the students:170 160 180 150 165 175
Output:
Sorted heights of students:150 160 165 170 175 180
Time taken for sorting: 0.000015 seconds
Time Complexity:

- **Best and Average Case**: O(n log n)
- **Worst Case** (if the pivot is always the smallest or largest element): $O(n^2)$

Notes:

- Quick Sort is very efficient for large datasets if the pivot selection is well-optimized (e.g., using the median of three strategy).
- The time measurement in the code will depend on the size of the array and the performance of the system on which it's run.

3. Print all the nodes reachable from a given starting node in agraph using Depth First Search method and Breadth First Search. Also check whether a graph is connected.

Description of Depth First Search (DFS) and Breadth First Search (BFS):
Depth First Search (DFS): DFS is an algorithm for traversing or searching through a graph. Starting from a given node, it explores as far as possible along each branch before backtracking. It uses a stack (either explicitly or via recursion) to keep track of the vertices that need to be explored. DFS is particularly useful for exploring the structure of a graph in-depth.
Breadth First Search (BFS): BFS is another algorithm for traversing a graph, but instead of exploring as far as possible along one branch like DFS, BFS explores all the neighboring nodes level by level, starting from the given node. It uses a queue to keep track of the nodes to visit next.
Connected Graph: A graph is said to be connected if there is a path between every pair of nodes. In a connected graph, starting from any node, it is possible to visit every other node via the edges of the graph.

```
#include <stdio.h>
#include <stdlib.h>
#define MAX_NODES 100
// Graph representation
int graph[MAX_NODES][MAX_NODES];
int visited[MAX_NODES];
int num_nodes;
```

```
// Function to perform DFS
void DFS(int node) {
    visited[node] = 1;
    printf("%d ", node);

    // Visit all the adjacent nodes of the current node
    for (int i = 0; i < num_nodes; i++) {
        if (graph[node][i] == 1 && !visited[i]) {
            DFS(i);
        }
    }
}
// Function to perform BFS
void BFS(int start) {
    int queue[MAX_NODES], front = 0, rear = 0;
    visited[start] = 1;
    queue[rear++] = start;
    printf("%d ", start);

    while (front < rear) {
        int current = queue[front++];

        // Visit all adjacent nodes of the current node
        for (int i = 0; i < num_nodes; i++) {
            if (graph[current][i] == 1 && !visited[i]) {
                visited[i] = 1;
                queue[rear++] = i;
                printf("%d ", i);
            }
        }
    }
}
// Function to check if the graph is connectedint isConnected() {
    // Reset visited array
    for (int i = 0; i < num_nodes; i++) {
        visited[i] = 0;
    }

    // Start DFS or BFS from node 0
    DFS(0);

    // Check if all nodes were visited
    for (int i = 0; i < num_nodes; i++) {
        if (visited[i] == 0) {
            return 0;  // Graph is not connected
        }
    }
    return 1;  // Graph is connected
}
int main() {
```

```
    int edges, u, v;

    // Input number of nodes and edges
    printf("Enter the number of nodes: ");
    scanf("%d", &num_nodes);

    // Initialize graph to 0 (no edges)
    for (int i = 0; i < num_nodes; i++) {
        for (int j = 0; j < num_nodes; j++) {
            graph[i][j] = 0;
        }
    }

    // Input number of edges
    printf("Enter the number of edges: ");
    scanf("%d", &edges);

    // Input edges (u, v) representing an undirected graph
    printf("Enter the edges (u, v):\n");
    for (int i = 0; i < edges; i++) {
        scanf("%d %d", &u, &v);
        graph[u][v] = 1;
        graph[v][u] = 1;  // Since the graph is undirected
    }

    // Perform DFS starting from node 0
    printf("DFS traversal starting from node 0:\n");
    for (int i = 0; i < num_nodes; i++) visited[i] = 0;  // Reset visited array
    DFS(0);
    printf("\n");

    // Perform BFS starting from node 0
    printf("BFS traversal starting from node 0:\n");
    for (int i = 0; i < num_nodes; i++) visited[i] = 0;  // Reset visited array
    BFS(0);
    printf("\n");

    // Check if the graph is connected
    for (int i = 0; i < num_nodes; i++) visited[i] = 0;  // Reset visited array
    if (isConnected()) {
        printf("The graph is connected.\n");
    } else {
        printf("The graph is not connected.\n");
    }

    return 0;
}
```

Explanation of the Code:

1. **Graph Representation**: The graph is represented as an adjacency matrix graph[MAX_NODES][MAX_NODES], where each entry graph[i][j] is 1 if there is an edge between nodes i and j, and 0 otherwise.
2. **DFS Function**:
    1. The DFS function explores the graph recursively. It starts from a given node, marks it as visited, and recursively visits all unvisited neighboring nodes.
3. **BFS Function**:

    1. The BFS function uses a queue to explore the graph level by level. It starts from a given node, marks it as visited, and visits all unvisited neighboring nodes.

2. **isConnected Function**:

    1. This function checks if the graph is connected by performing DFS from node 0 and ensuring all nodes are visited. If any node is not visited, the graph is not connected.

2. **Main Function**:

    1. It accepts the number of nodes and edges as input, initializes the graph, performs DFS and BFS traversals starting from node 0, and checks if the graph is connected.

Example Input/Output:
Input:
Enter the number of nodes: 5
Enter the number of edges: 4
Enter the edges (u, v):0 10 21 33 4
Output:
DFS traversal starting from node 0:0 1 3 4 2
BFS traversal starting from node 0:0 1 2 3 4
The graph is connected.
Explanation of the Input/Output:

- The graph has 5 nodes and 4 edges.
- The graph is connected since there is a path between every pair of nodes.
- The DFS and BFS methods print the order in which nodes are visited.
- The graph is checked for connectivity and found to be connected.

Time Complexity:

- **DFS and BFS** both have a time complexity of **O(V + E)**, where V is the number of vertices and E is the number of edges.
- Checking for connectivity by DFS also takes **O(V + E)** time.

4. **Obtain the topological ordering of vertices in a given digraph.**

Description of Topological Ordering:
Topological ordering (or topological sort) is a linear ordering of vertices in a directed graph such that for every directed edge u→vu \to vu→v, vertex uuu comes before vvv in the ordering. This is only possible for Directed Acyclic Graphs (DAGs), meaning the graph must not contain any cycles.
Topological sorting is particularly useful in scenarios such as:

- Task scheduling, where tasks must be performed in a particular order based on dependencies.
- Resolving symbol dependencies in compilers.
- Managing workflow processes.

Key Characteristics of Topological Sorting:

1. **Unique Ordering**: A DAG may have multiple valid topological orderings.
2. **Cyclic Graphs**: A graph with cycles cannot have a topological ordering.

Algorithm for Topological Sort:

- **Kahn's Algorithm** (using in-degree method) is one popular method for topological sorting:

    1. Compute the in-degree (number of incoming edges) for each vertex.
    2. Put all vertices with zero in-degree into a queue.
    3. Repeatedly remove vertices from the queue and add them to the topological order.
    4. For each adjacent vertex, decrease its in-degree. If it becomes zero, add it to the queue.
    5. If at the end, there are still vertices with non-zero in-degree, the graph contains a cycle, and topological sorting is not possible.

```
#include <stdio.h>
#include <stdlib.h>
#define MAX_NODES 100
// Graph representation
int graph[MAX_NODES][MAX_NODES];
int in_degree[MAX_NODES];int num_nodes;
// Function to perform topological sort using Kahn's algorithm
```

```
void topologicalSort() {
    int queue[MAX_NODES], front = 0, rear = 0;
    int top_order[MAX_NODES];
    int index = 0;

    // Initialize in-degree array
    for (int i = 0; i < num_nodes; i++) {
        in_degree[i] = 0;
    }

    // Calculate in-degree of each vertex
    for (int u = 0; u < num_nodes; u++) {
        for (int v = 0; v < num_nodes; v++) {
            if (graph[u][v] == 1) {
                in_degree[v]++;
            }
        }
    }

    // Enqueue vertices with zero in-degree
    for (int i = 0; i < num_nodes; i++) {
        if (in_degree[i] == 0) {
            queue[rear++] = i;
        }
    }

    // Process vertices in the queue
    while (front < rear) {
        int node = queue[front++];
        top_order[index++] = node;

        // Decrease the in-degree of adjacent vertices
        for (int i = 0; i < num_nodes; i++) {
            if (graph[node][i] == 1) {
                in_degree[i]--;
                if (in_degree[i] == 0) {
                    queue[rear++] = i;
                }
            }
        }
    }

    // If topological sort includes all nodes, print it
    if (index == num_nodes) {
        printf("Topological Sort Order:\n");
        for (int i = 0; i < num_nodes; i++) {
            printf("%d ", top_order[i]);
        }
        printf("\n");
    } else {
```

```
        printf("The graph has a cycle and cannot be topologically sorted.\n");
    }
}
int main() {
    int edges, u, v;

    // Input number of nodes and edges
    printf("Enter the number of nodes: ");
    scanf("%d", &num_nodes);

    // Initialize graph to 0 (no edges)
    for (int i = 0; i < num_nodes; i++) {
        for (int j = 0; j < num_nodes; j++) {
            graph[i][j] = 0;
        }
    }

    // Input number of edges
    printf("Enter the number of edges: ");
    scanf("%d", &edges);

    // Input edges (u, v) representing a directed graph
    printf("Enter the edges (u, v):\n");
    for (int i = 0; i < edges; i++) {
        scanf("%d %d", &u, &v);
        graph[u][v] = 1;  // Directed edge u -> v
    }

    // Perform topological sort
    topologicalSort();

    return 0;
}
```

Explanation of the Code:

Graph Representation: The graph is represented as an adjacency matrix graph[MAX_NODES][MAX_NODES]. Each entry graph[u][v] = 1 indicates a directed edge from node u to node v.

In-degree Array: An array in_degree[MAX_NODES] stores the in-degree (number of incoming edges) for each vertex. This is used to identify nodes that have no incoming edges, which are the starting points for the topological sort.

Kahn's Algorithm:

- **Step 1**: Initialize the in-degree for all vertices.
- **Step 2**: Add all vertices with zero in-degree to a queue.
- **Step 3**: Process each vertex from the queue:
    - Add it to the topological order.
    - Decrease the in-degree of its neighbors. If a neighbor's in-degree becomes zero, add it to the queue.

- **Step 4**: If all nodes are processed, the topological order is printed. If there are still vertices left with non-zero in-degree, it means there is a cycle, and topological sorting is not possible.

Cycle Detection: If not all vertices are included in the topological order, the graph contains a cycle.

Example Input/Output:
Input:
Enter the number of nodes: 6
Enter the number of edges: 6
Enter the edges (u, v):5 25 04 04 12 33 1
Output:
Topological Sort Order:5 4 2 3 1 0
Explanation:

- The graph contains six nodes and six directed edges.
- The topological sort order is a linear ordering of vertices that respects the direction of the edges.
- In this example, vertex 5 comes before 2 and 0, and vertex 4 comes before 0 and 1, etc.

Time Complexity:

- **Kahn's Algorithm** has a time complexity of **O(V + E)**, where V is the number of vertices and E is the number of edges. This is because we:
  - Compute the in-degree of each vertex (O(V + E)).
  - Process each vertex and edge exactly once (O(V + E)).

Notes:

- The program assumes the graph is represented as a directed graph using edges (u, v), where the edge goes from vertex u to vertex v.
- The program detects if the graph contains a cycle and prints an appropriate message.

5. Implement Horspool algorithm for String Matching.

Description of Horspool's Algorithm:
Horspool's algorithm is an efficient string matching algorithm used for finding occurrences of a pattern in a text. It is a variation of the **Boyer-Moore** algorithm, designed to be simpler and easier to implement. The basic idea behind Horspool's algorithm is to shift the pattern to the right when a mismatch is found, by utilizing a "bad character rule."

Bad Character Heuristic:

1. **Pattern Matching**: Start by aligning the pattern with the beginning of the text.
2. **Matching Process**: Compare the pattern with the text from right to left (starting from the last character of the pattern).
3. **Mismatch Handling**: If a mismatch occurs, use a "shift table" to determine how many positions the pattern should be shifted to the right. The shift is determined by the "bad character rule":
    1. For each mismatch, the table provides the number of positions to shift based on the last character of the pattern that matched with the current text character.
4. **Shift**: The pattern is shifted so that the next occurrence of the mismatched character aligns with the last occurrence of that character in the pattern (if the character exists in the pattern).

Steps of Horspool's Algorithm:

1. **Preprocessing**: Create a "shift table" (or bad character rule table) for the pattern. This table tells you how many positions you need to shift the pattern for each character in the pattern.
2. **Pattern Search**: Use the shift table to perform the pattern search in the text.

```
#include <stdio.h>
#include <string.h>
#define MAX_CHARACTERS 256  // Assuming ASCII characters
// Function to preprocess the pattern and create the shift tablevoid
createShiftTable(char* pattern, int m, int shiftTable[])
 {
   // Initialize all values of the shift table to the length of the pattern
   for (int i = 0; i < MAX_CHARACTERS; i++) {
      shiftTable[i] = m;
   }

   // Update the shift table based on the pattern
   for (int i = 0; i < m - 1; i++) {
      shiftTable[(int)pattern[i]] = m - i - 1;
   }
}
// Horspool's Algorithm for string matchingint horspoolSearch(char* text, char*
pattern) {
   int n = strlen(text);
   int m = strlen(pattern);

   // Shift table for the pattern
   int shiftTable[MAX_CHARACTERS];

   // Create the shift table
   createShiftTable(pattern, m, shiftTable);

   int i = m - 1;  // Start from the end of the pattern
```

```
    int j = m - 1;  // Pattern index

    while (i < n) {
        // Compare the pattern with the text starting from right to left
        if (text[i] == pattern[j]) {
            // If there's a match, check if the entire pattern matches
            if (j == 0) {
                // Pattern found at index (i)
                return i;
            }
            i--;
            j--;
        } else {
            // Mismatch, shift the pattern using the shift table
            i += shiftTable[(int)text[i]];
            j = m - 1;
        }
    }

    // Return -1 if no match is found
    return -1;
}
int main() {
    char text[100], pattern[100];

    // Input text and pattern
    printf("Enter the text: ");
    scanf("%s", text);

    printf("Enter the pattern: ");
    scanf("%s", pattern);

    // Perform Horspool's search
    int result = horspoolSearch(text, pattern);

    if (result != -1) {
        printf("Pattern found at index %d\n", result);
    } else {
        printf("Pattern not found\n");
    }

    return 0;
}
```

Explanation of the Code:

Preprocessing Step (Create Shift Table):

1. The createShiftTable function generates a "shift table" for the pattern. This table is created by initializing the shift value for each character (from the ASCII set) to the length of the pattern. Then, for each character in the pattern (except the last one), the shift table is updated

to store how far we need to shift the pattern when that character is encountered in the text.

Horspool's Search:

1. The horspoolSearch function starts searching the text from the rightmost side of the pattern. If a mismatch occurs, the shift table is used to determine how far the pattern should be shifted to the right.
2. If a match is found, it checks if the entire pattern matches and returns the index where the pattern was found in the text.
3. If no match is found, it returns -1.

Main Function:

1. Takes the text and the pattern as input.
2. Calls the horspoolSearch function to find the pattern in the text.
3. Displays the result (index of the first occurrence of the pattern or a "not found" message).

Example Input/Output:
Input:
Enter the text: ABABABABCAB
Enter the pattern: ABC
Output:
Pattern found at index 7
Explanation:

- The algorithm searches for the pattern "ABC" in the text "ABABABABCAB" and finds it starting at index 7.

Time Complexity:

- **Preprocessing Time**: Creating the shift table takes **O(m)** time, where m is the length of the pattern.
- **Search Time**: The search takes **O(n)** time, where n is the length of the text. In the worst case, every character in the text is compared once, but the shift table helps skip unnecessary comparisons, making it efficient.

Therefore, the overall time complexity of Horspool's algorithm is **O(n + m)**, where n is the length of the text and m is the length of the pattern.
Space Complexity:

- **Shift Table**: The space complexity is **O(k)**, where k is the size of the character set (in this case, 256 for ASCII characters).
- **Auxiliary Space**: The program uses a constant amount of space for variables and the input/output, so space complexity is **O(k)**.

Conclusion:

Horspool's algorithm is an efficient string matching algorithm based on the Boyer-Moore algorithm's bad character rule, with a simpler implementation. It is suitable for many practical applications where string matching is required.

6.Sort a given set of elements using the Heap sort method.

Heap Sort Overview

Heap Sort is a comparison-based sorting algorithm that works by utilizing a binary heap data structure. The idea behind Heap Sort is to first build a **max-heap** (for ascending order) or **min-heap** (for descending order) from the input data. A max-heap is a binary tree where the value of each node is greater than or equal to the values of its children.

Once the max-heap is built, the root element (which is the largest) is swapped with the last element in the array, and the heap is reduced in size. The process is repeated for the remaining heap elements. This process continues until the entire array is sorted.

Steps for Heap Sort

1. **Build a Max-Heap** from the input data.
2. **Swap the root element** (the maximum) with the last element in the heap.
3. **Reduce the heap size** by 1.
4. **Reheapify** the heap to restore the heap property.
5. **Repeat** steps 2 to 4 until the heap size becomes 1.

```
#include <stdio.h>
// Function to heapify a subtree rooted with node i
void heapify(int arr[], int n, int i)
{
 int largest = i; // Initialize largest as root
int left = 2 * i + 1; // Left child
int right = 2 * i + 2; // Right child
// If left child is larger than root
if (left < n && arr[left] > arr[largest])
{
 largest = left;
 }
```

```
// If right child is larger than the largest so far
if (right < n && arr[right] > arr[largest])
{ largest = right; }
// If largest is not root
if (largest != i)
{ // Swap the elements
int temp = arr[i];
fected subtree heapify(arr, n, largest);
 }
 } // Function to implement heap sort
void heapSort(int arr[], int n)
 { // Build a max-heap
 for (int i = n / 2 - 1; i >= 0; i--)
 { heapify(arr, n, i); }
// One by one extract elements from the heap
for (int i = n - 1; i > 0; i--)
 { // Move current root to the end
int temp = arr[0];
arr[0] = arr[i];
 arr[i] = temp; // Call heapify on the reduced heap
eapify(arr, i, 0); } } // Function to print the array
 void printArray(int arr[], int size)
{
for (int i = 0; i < size; i++)
{
printf("%d ", arr[i]); }
 printf("\n");
 }

int main()
{
int n;
// Taking input from the user
printf("Enter the number of elements: ");
scanf("%d", &n);
 int arr[n];
// Declaring the array with size 'n'
printf("Enter the elements:\n");
for (int i = 0; i < n; i++)
{ scanf("%d", &arr[i]); }
 printf("Original array: \n");
printArray(arr, n);
heapSort(arr, n);
printf("Sorted array: \n");
printArray(arr, n);
return 0;
}
```

Explanation of the Code

heapify function: This function ensures the heap property is maintained by recursively ensuring the largest element is at the root. It compares the node with its children and swaps the values if needed.

heapSort function: This function first builds a max-heap by calling heapify on each node starting from the middle of the array. Then, it repeatedly swaps the root of the heap (which is the largest element) with the last element and reduces the heap size by one. After swapping, heapify is called on the root to restore the heap property.

printArray function: This simply prints the array to verify the sorting result.

Sample Output

Enter the number of elements: 6
Enter the elements: 12 11 13 5 6 7
Original array: 12 11 13 5 6 7
Sorted array: 5 6 7 11 12 13

Time Complexity

- **Building the heap**: O(n)
- **Heap sort**: O(n log n) for both average and worst cases

Thus, Heap Sort is an efficient sorting algorithm with a time complexity of O(n log n) and does not require additional space (other than the input array), making it an in-place sorting algorithm.

窗体顶端

窗体底端

6. **Implement Floyd's algorithm and Warshall's algorithm for a given graph.**

Both **Floyd's Algorithm** and **Warshall's Algorithm** are used to find the **shortest paths** and **transitive closures** in a graph, respectively.

Floyd's Algorithm (Floyd-Warshall Algorithm):

1. This algorithm is used to find the shortest paths between all pairs of vertices in a weighted graph.
2. It works for both directed and undirected graphs.
3. It can handle negative weight edges, but it cannot handle negative weight cycles.
4. The time complexity is $O(V^3)$, where V is the number of vertices in the graph.

Warshall's Algorithm:

1. This algorithm is used to find the **transitive closure** of a directed graph. The transitive closure of a graph tells us whether there is a path between two vertices.
2. The time complexity is $O(V^3)$, where V is the number of vertices.

The C program below implements both Floyd's algorithm for shortest paths and Warshall's algorithm for transitive closure on a graph represented by an adjacency matrix.

```
#include <stdio.h>
#define INF 99999  // Define a large number for infinity
// Function to implement Floyd's Algorithm
void floydWarshall(int graph[][10], int V)
{
   int dist[V][V], i, j, k;

   // Initialize the distance matrix same as the input graph matrix
      for (i = 0; i < V; i++)
      {
      for (j = 0; j < V; j++) {
         if (graph[i][j] == 0 && i != j)
            dist[i][j] = INF;
         else
            dist[i][j] = graph[i][j];
      }
   }

   // Floyd-Warshall algorithm to compute shortest paths
   for (k = 0; k < V; k++) {
      for (i = 0; i < V; i++) {
         for (j = 0; j < V; j++) {
            if (dist[i][k] + dist[k][j] < dist[i][j]) {
               dist[i][j] = dist[i][k] + dist[k][j];
            }
         }
      }
   }

   // Print the shortest distance matrix
   printf("\nShortest distances between every pair of vertices:\n");
   for (i = 0; i < V; i++) {
```

```
        for (j = 0; j < V; j++) {
            if (dist[i][j] == INF)
                printf("INF ");
            else
                printf("%d ", dist[i][j]);
        }
        printf("\n");
    }
}
// Function to implement Warshall's Algorithm (Transitive Closure)void warshall(int
graph[][10], int V) {
    int reach[V][V], i, j, k;

    // Initialize the reachability matrix same as the input graph matrix
    for (i = 0; i < V; i++) {
        for (j = 0; j < V; j++) {
            reach[i][j] = graph[i][j];
        }
    }

    // Warshall's algorithm to compute the transitive closure
    for (k = 0; k < V; k++) {
        for (i = 0; i < V; i++) {
            for (j = 0; j < V; j++) {
                if (reach[i][k] && reach[k][j]) {
                    reach[i][j] = 1;
                }
            }
        }
    }

    // Print the reachability matrix
    printf("\nTransitive closure (reachability) matrix:\n");
    for (i = 0; i < V; i++) {
        for (j = 0; j < V; j++) {
            printf("%d ", reach[i][j]);
        }
        printf("\n");
    }
}
int main() {
    int V, i, j;

    // Taking input for the number of vertices
    printf("Enter the number of vertices: ");
    scanf("%d", &V);
    int graph[V][V];

    // Input the adjacency matrix for the graph
```

```
    printf("Enter the adjacency matrix (use 0 for no edge, use positive numbers for
edge weights):\n");
    for (i = 0; i < V; i++) {
        for (j = 0; j < V; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    // Run Floyd's Algorithm for shortest paths
    floydWarshall(graph, V);

    // Run Warshall's Algorithm for transitive closure
    warshall(graph, V);
    return 0;
}
```

Explanation of the Code

Input:

1. The number of vertices (V) is input by the user.
2. The adjacency matrix is then entered by the user. For a graph with no edge between two vertices, the input value should be 0. For directed graphs, an edge weight is the number itself.
3. If an edge is missing, 0 is used, and for the purposes of Floyd's algorithm, it's treated as infinity (except the diagonal which represents self-loops).

Floyd's Algorithm:

1. We initialize the distance matrix to the input graph. If there's no direct edge between two nodes, we assign INF to represent an infinite distance.
2. The algorithm then computes the shortest paths between all pairs of vertices by considering each vertex as an intermediate point.
3. Finally, the shortest paths are printed.

Warshall's Algorithm:

1. This algorithm calculates the transitive closure by determining if there is a path between every pair of vertices in the graph.
2. The result is printed as a reachability matrix.

Time Complexity:

1. Both Floyd-Warshall and Warshall's algorithms have a time complexity of **$O(V^3)$**, where V is the number of vertices.

Sample Output

Enter the number of vertices: 4
Enter the adjacency matrix (use 0 for no edge, use positive numbers for edge weights):

0 3 0 0
0 0 1 0
0 0 0 2
0 0 0 0

Shortest distances between every pair of vertices:
0 3 4 5
7 0 1 2
3 6 0 2
5 8 3 0

Transitive closure (reachability) matrix:
1 1 1 1
0 1 1 1
0 0 1 1
0 0 0 1

Explanation of Output:

- The **Shortest distances** matrix shows the minimum distance between every pair of vertices in the graph.
- The **Transitive closure** matrix indicates whether there is a path between any two vertices. A 1 means a path exists, and a 0 means no path exists.

Both Floyd's and Warshall's algorithms provide useful tools for graph analysis and pathfinding. Floyd's algorithm computes shortest paths, while Warshall's algorithm determines connectivity in the graph.

8.There are n different routes from hostel to college. Each route incurs some cost. Find the minimum cost route to reach the college from the hostel using Prim's algorithm.

Prim's Algorithm is a **greedy algorithm** used to find the **Minimum Spanning Tree (MST)** of a weighted, connected graph. It ensures that the sum of the weights of the edges in the spanning tree is minimized. The algorithm starts from any arbitrary vertex and grows the MST by adding edges with the minimum possible cost that connect new vertices to the growing tree.

For the problem of finding the minimum cost route from the hostel to the college among n different routes, we can represent the routes as a graph where:

- Each route between the hostel and college is an edge with a cost.
- The algorithm will find the minimum cost route from hostel to college, which is essentially finding the Minimum Spanning Tree (MST) of the graph.

Steps in Prim's Algorithm

1. Initialize a set of vertices (starting with any one vertex).
2. Mark the starting vertex as included in the MST.
3. Repeatedly add the least expensive edge that connects a new vertex to the MST, ensuring no cycles are formed.
4. Continue until all vertices are included in the MST.

C Program for Prim's Algorithm

The following C program implements Prim's algorithm to find the minimum cost route in a graph:

```
#include <stdio.h>
#include <stdbool.h>
#define INF 999999  // Define a large number for infinity
// Function to find the vertex with the minimum key value
int minKey(int key[], bool mstSet[], int V) {
   int min = INF, min_index;

   for (int v = 0; v < V; v++) {
      if (mstSet[v] == false && key[v] < min) {
         min = key[v];
         min_index = v;
      }
   }

   return min_index;
}
// Function to implement Prim's algorithm to find the minimum cost routevoid
primMST(int graph[][10], int V) {
   int parent[V];  // Array to store the constructed MST
   int key[V];     // Key values to pick minimum weight edge
   bool mstSet[V]; // To represent the set of vertices included in MST

   // Initialize all keys to infinity, MST set to false, and parent to -1
   for (int i = 0; i < V; i++) {
      key[i] = INF;
      mstSet[i] = false;
      parent[i] = -1;
   }

   // Start from the first vertex
   key[0] = 0;  // Make the key value of the starting vertex 0

   // Find the MST of the graph
   for (int count = 0; count < V - 1; count++) {
      // Pick the minimum key vertex from the set of vertices not yet included in MST
      int u = minKey(key, mstSet, V);

      // Add the picked vertex to the MST set
      mstSet[u] = true;
```

```
        // Update key value and parent index of the adjacent vertices of the picked vertex
        for (int v = 0; v < V; v++) {
            // Update the key and parent if the vertex v is not in MST and there is an edge
from u to v
            if (graph[u][v] && mstSet[v] == false && graph[u][v] < key[v]) {
                key[v] = graph[u][v];
                parent[v] = u;
            }
        }
    }

    // Print the constructed MST
    printf("Edge   Weight\n");
    int total_cost = 0;
    for (int i = 1; i < V; i++) {
        printf("%d - %d   %d\n", parent[i], i, graph[i][parent[i]]);
        total_cost += graph[i][parent[i]];
    }
    printf("\nTotal minimum cost: %d\n", total_cost);
}
int main() {
    int V;

    // Taking input for number of vertices (routes)
    printf("Enter the number of routes (vertices): ");
    scanf("%d", &V);

    int graph[V][V];

    // Taking input for the adjacency matrix (costs between the routes)
     printf("Enter the cost matrix for the routes (use 0 for no direct route, other values
for the cost):\n");
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    // Run Prim's algorithm to find the minimum cost route
    primMST(graph, V);

    return 0;
}
```

Explanation of the Code

Input:

- The number of routes (vertices) V is input by the user.
- The adjacency matrix representing the cost between routes is input. Each entry graph[i][j] indicates the cost of traveling from route i to

route j. If no direct route exists between two routes, the cost should be set to 0 (or a large number can be used to represent infinity).

Prim's Algorithm:

- **Key Array**: The key[] array holds the minimum edge weight to connect each vertex to the MST.
- **MST Set**: The mstSet[] array tracks whether a vertex is included in the MST.
- **Parent Array**: The parent[] array stores the MST structure, i.e., which vertex each vertex is connected to in the MST.
- **minKey Function**: This function selects the vertex with the minimum key value that hasn't been included in the MST yet.

MST Construction:

- The algorithm starts from any vertex (here, vertex 0).
- It repeatedly picks the vertex with the smallest edge weight that is not in the MST and adds it to the MST.
- After all vertices are included, the MST is printed with the edges and their costs.

Output:

- The program prints the edges of the MST along with their respective costs.
- It also prints the total minimum cost of the spanning tree (which represents the minimum cost to travel through all routes).

Sample Output

Enter the number of routes (vertices): 5
Enter the cost matrix for the routes (use 0 for no direct route, other values for the cost):
0 2 0 6 0
2 0 3 8 5
0 3 0 0 7
6 8 0 9 0
0 5 7 0 9

Edge Weight
0 - 1 2
1 - 2 3
0 - 3 6
1 - 4 5

Total minimum cost: 16

Explanation of Output:

- The program outputs the edges of the Minimum Spanning Tree (MST), with the corresponding costs.

- The total cost of the MST is 16, which is the minimum cost to cover all routes between the hostel and the college.

Time Complexity:

- **Time complexity** of Prim's algorithm is **$O(V^2)$**, where V is the number of vertices (routes), primarily due to the need to check all edges for each vertex.

9. Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm

Kruskal's algorithm is a **greedy algorithm** used to find the **Minimum Spanning Tree (MST)** of a connected, undirected graph. The algorithm follows a straightforward approach where edges are considered one by one and added to the MST, ensuring no cycles are formed and the edges selected are the smallest possible.
The algorithm works as follows:

1. Sort all the edges of the graph in non-decreasing order of their weight.
2. Pick the smallest edge. Check if including this edge forms a cycle with the MST formed so far (using a **disjoint-set** or **union-find** data structure).
3. If including the edge doesn't form a cycle, add it to the MST.
4. Repeat the process until we have selected V-1 edges (where V is the number of vertices).

Steps in Kruskal's Algorithm

1. Sort all edges in the graph based on their weight.
2. Initialize a disjoint-set data structure to keep track of the connected components of the graph.
3. For each edge in the sorted list:

    1. Check if the two vertices of the edge belong to different sets.
    2. If they belong to different sets, add the edge to the MST and combine the two sets.

4. Repeat until we have V-1 edges in the MST.

The following C program implements Kruskal's algorithm to find the Minimum Cost Spanning Tree (MST) of a given undirected graph:

```
#include <stdio.h>
#include <stdlib.h>
#define MAX_EDGES 100
#define INF 999999
// Structure for edges
typedef struct {
    int u, v, weight;
} Edge;
// Structure for the disjoint-set (Union-Find)
typedef struct {
    int parent[MAX_EDGES];
    int rank[MAX_EDGES];
} DisjointSet;
// Function to initialize the disjoint-set
void initSet(DisjointSet *ds, int V) {
    for (int i = 0; i < V; i++) {
        ds->parent[i] = i;
        ds->rank[i] = 0;
    }
}
// Function to find the root of a vertex
int find(DisjointSet *ds, int x) {
    if (ds->parent[x] != x)
        ds->parent[x] = find(ds, ds->parent[x]);  // Path compression
    return ds->parent[x];
}
// Function to perform union of two sets
void unionSets(DisjointSet *ds, int x, int y) {
    int rootX = find(ds, x);
    int rootY = find(ds, y);

    if (rootX != rootY) {
        // Union by rank
        if (ds->rank[rootX] > ds->rank[rootY]) {
            ds->parent[rootY] = rootX;
        } else if (ds->rank[rootX] < ds->rank[rootY]) {
            ds->parent[rootX] = rootY;
        } else {
            ds->parent[rootY] = rootX;
            ds->rank[rootX]++;
        }
    }
}
// Function to compare two edges (used for sorting edges)int compare(const void *a,
const void *b) {
    return ((Edge*)a)->weight - ((Edge*)b)->weight;
}
// Function to implement Kruskal's Algorithmvoid kruskal(Edge edges[], int E, int V)
{
    DisjointSet ds;
```

```
    initSet(&ds, V);  // Initialize the disjoint-set

    // Sort the edges by weight
    qsort(edges, E, sizeof(Edge), compare);

    int mstWeight = 0;
    int mstEdges = 0;

    printf("Edges in the Minimum Spanning Tree (MST):\n");

    for (int i = 0; i < E; i++) {
        int u = edges[i].u;
        int v = edges[i].v;
        int weight = edges[i].weight;

        // If u and v are in different sets, include this edge in the MST
        if (find(&ds, u) != find(&ds, v)) {
            unionSets(&ds, u, v);  // Combine the sets
            printf("%d - %d: %d\n", u, v, weight);  // Print the edge
            mstWeight += weight;  // Add the weight of the edge
            mstEdges++;

            // If we have found V-1 edges, the MST is complete
            if (mstEdges == V - 1) {
                break;
            }
        }
    }

    printf("\nTotal weight of MST: %d\n", mstWeight);
}
int main() {
    int V, E;

    // Taking input for the number of vertices and edges
    printf("Enter the number of vertices: ");
    scanf("%d", &V);

    printf("Enter the number of edges: ");
    scanf("%d", &E);

    Edge edges[E];

    // Taking input for the edges (u, v, weight)
    printf("Enter the edges (u, v, weight):\n");
    for (int i = 0; i < E; i++) {
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].weight);
    }

    // Running Kruskal's algorithm to find the MST
```

```
    kruskal(edges, E, V);

    return 0;
}
```

Explanation of the Code

Input:

1. The program first takes the number of vertices (V) and edges (E) as input.
2. It then takes the details of each edge in the form (u, v, weight), where u and v are the endpoints of the edge, and weight is the cost associated with the edge.

Disjoint-Set Data Structure:

1. **Find**: This function finds the root of a given vertex using path compression to speed up future queries.
2. **Union**: This function merges two sets using **union by rank**, which helps keep the tree shallow and improves the efficiency of subsequent operations.

Kruskal's Algorithm:

1. The edges are sorted in non-decreasing order of their weights.
2. The algorithm iterates through the sorted edges and adds the edge to the MST if it connects two different sets (i.e., it does not form a cycle).
3. The union function is used to combine the sets of the two vertices of the edge.

Output:

1. The program prints the edges included in the MST along with their weights.
2. The total weight of the MST is also displayed at the end.

Sample Output

```
Enter the number of vertices: 4
Enter the number of edges: 5
Enter the edges (u, v, weight):
0 1 1
0 0 2
6 0 3
5 3 15
2 3 4
Edges in the Minimum Spanning Tree (MST):
2 - 3: 4
0 - 3: 5
0 - 1: 10
Total weight of MST: 19
```

Explanation of the Output:

- The **Minimum Spanning Tree (MST)** is formed by the edges (2-3), (0-3), and (0-1).

- The total weight of the MST is 19, which is the minimum cost required to connect all vertices in the graph.

Time Complexity:

- Sorting the edges takes **O(E log E)** time.
- The operations related to union and find are very efficient, with an almost constant time complexity due to the union by rank and path compression techniques, effectively making it **O(E α(V))**, where α is the inverse Ackermann function.
- Therefore, the overall time complexity is **O(E log E)**, which is efficient for most graphs.

10. mplement 0/1 Knapsack problem using dynamic programming.

The **0/1 Knapsack Problem** is a well-known optimization problem where we are given:

- A set of items, each with a weight and a value.
- A knapsack that can hold a limited weight capacity.

The goal is to determine the maximum value that can be obtained by selecting items such that their total weight does not exceed the knapsack's weight capacity.
In the **0/1 Knapsack**, each item can either be selected or not selected (i.e., it's either included or excluded from the knapsack).
Dynamic Programming Approach
The 0/1 Knapsack problem can be solved efficiently using **dynamic programming**. The idea is to build a solution incrementally by solving smaller subproblems and storing the results to avoid redundant calculations. This approach helps reduce the time complexity from exponential to polynomial.
The approach works as follows:

1. Let dp[i][w] represent the maximum value achievable using the first i items with a knapsack of capacity w.
2. The recurrence relation is:
    - If the item i is not included: dp[i][w] = dp[i-1][w]
    - If the item i is included: dp[i][w] = dp[i-1][w-weight[i]] + value[i]
    - The solution is the maximum of these two values.
3. We construct the table dp[][] to solve the problem and extract the result from dp[n][W], where n is the number of items and W is the capacity of the knapsack.

C Program for 0/1 Knapsack Problem
Here is the C program that implements the **0/1 Knapsack Problem** using dynamic programming:
```
#include <stdio.h>
// Function to calculate the maximum value of the knapsack
```

```
int knapsack(int W, int wt[], int val[], int n)
{
   int dp[n+1][W+1];  // DP table to store the maximum value for each subproblem

   // Build the DP table
   for (int i = 0; i <= n; i++) {
      for (int w = 0; w <= W; w++) {
         if (i == 0 || w == 0) {
         dp[i][w] = 0;  // Base case: no items or zero capacity
         } else if (wt[i-1] <= w) {
// If the item can be included, take the maximum of including or excluding it
dp[i][w] = (val[i-1] + dp[i-1][w-wt[i-1]] > dp[i-1][w]) ?
               (val[i-1] + dp[i-1][w-wt[i-1]]) : dp[i-1][w];
         }
 else {
      dp[i][w] = dp[i-1][w];  // Item can't be included
         }
      }
   }

   // The value in dp[n][W] is the maximum value that can be obtained
   return dp[n][W];
}
int main()
 {
   int n, W;

   // Taking input for number of items and the capacity of the knapsack
   printf("Enter the number of items: ");
   scanf("%d", &n);
   printf("Enter the knapsack capacity: ");
   scanf("%d", &W);

   int val[n], wt[n];

   // Taking input for values and weights of the items
   printf("Enter the values and weights of the items:\n");
   for (int i = 0; i < n; i++) {
      printf("Item %d: ", i + 1);
      scanf("%d %d", &val[i], &wt[i]);
   }
   // Calculate and print the maximum value that can be obtained
   int max_value = knapsack(W, wt, val, n);
   printf("Maximum value in Knapsack: %d\n", max_value);
   return 0;
}
```

Explanation of the Code

Input:

- The user inputs the number of items n and the knapsack's capacity W.

- The values and weights of the items are then entered by the user. Each item has a corresponding value and weight.

DP Table Construction:

- The program constructs a 2D DP table dp[i][w], where:
  - i represents the number of items considered so far.
  - w represents the current knapsack capacity being considered.
- The value dp[i][w] stores the maximum value achievable with the first i items and a knapsack of capacity w.

Base Case:

- When either the number of items (i) or the capacity (w) is 0, the maximum value is 0 because no items can be included.

Recurrence Relation:

- If the weight of the current item is less than or equal to the current knapsack capacity, we check whether including the item yields a better solution than excluding it:
  - dp[i][w] = max(val[i-1] + dp[i-1][w-wt[i-1]], dp[i-1][w])
- If the item cannot be included (its weight exceeds the current capacity), we exclude it:
  - dp[i][w] = dp[i-1][w]

Result:

- After filling the DP table, the maximum value that can be obtained with a knapsack of capacity W is found at dp[n][W].

Sample Output
Enter the number of items: 4
Enter the knapsack capacity: 7
Enter the values and weights of the items:
Item 1: 1 1
Item 2: 4 3
Item 3: 5 4
Item 4: 7 5
Maximum value in Knapsack: 9
Explanation of the Output:

- We have 4 items with the following values and weights:

- Item 1: Value = 1, Weight = 1
- Item 2: Value = 4, Weight = 3
- Item 3: Value = 5, Weight = 4
- Item 4: Value = 7, Weight = 5

- The knapsack capacity is 7. The maximum value that can be obtained by selecting items without exceeding the capacity is 9 (by including Item 2 and Item 3, for example).

Time Complexity:

- **Time Complexity**: The time complexity of the algorithm is **O(n * W)**, where n is the number of items and W is the knapsack's capacity.
- **Space Complexity**: The space complexity is **O(n * W)** due to the 2D DP table used to store the results of subproblems.

11. Implement N Queen's problem using Backtracking.

The **N Queen's Problem** is a classic problem in computer science and mathematics. The goal is to place N chess queens on an N x N chessboard such that no two queens threaten each other. This means that no two queens can share the same row, column, or diagonal.

Backtracking Approach

Backtracking is a systematic method of solving problems by trying all possible solutions and abandoning (or "backtracking") the ones that fail to satisfy the problem's constraints.

For the **N Queen's Problem**, the approach is as follows:

1. Start with an empty N x N chessboard.

2. Try to place a queen in a row, one column at a time.
3. For each column, check if the queen can be placed safely (i.e., it should not be attacked by any other queen already placed).
4. If the queen can be placed safely, recursively place queens in the subsequent rows.
5. If placing a queen leads to a solution, return true. If a conflict arises, backtrack and try placing the queen in a different column.
6. Continue this process until all queens are placed successfully or no solution is found.

Here is a C program that implements the **N Queen's Problem** using backtracking:

```
#include <stdio.h>
#include <stdbool.h>
#define MAX 100
// Function to print the chessboardvoid printSolution(int board[MAX][MAX], int N) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            printf(" %d ", board[i][j]);
        }
        printf("\n");
    }
}
// Function to check if a queen can be placed on board[row][col]bool isSafe(int board[MAX][MAX], int row, int col, int N) {
    // Check the column
    for (int i = 0; i < row; i++) {
        if (board[i][col] == 1) {
            return false;  // Another queen in the same column
        }
    }

    // Check the upper left diagonal
    for (int i = row, j = col; i >= 0 && j >= 0; i--, j--) {
        if (board[i][j] == 1) {
            return false;  // Another queen in the upper left diagonal
        }
    }

    // Check the upper right diagonal
    for (int i = row, j = col; i >= 0 && j < N; i--, j++) {
        if (board[i][j] == 1) {
            return false;  // Another queen in the upper right diagonal
        }
    }

    return true;
}
// Function to solve N Queen problem using backtrackingbool solveNQueens(int board[MAX][MAX], int row, int N) {
    // If all queens are placed
```

```
    if (row >= N) {
        return true;
    }

    // Try placing the queen in all columns for the current row
    for (int col = 0; col < N; col++) {
        // Check if the queen can be placed on board[row][col]
        if (isSafe(board, row, col, N)) {
            // Place the queen
            board[row][col] = 1;

            // Recur to place the rest of the queens
            if (solveNQueens(board, row + 1, N)) {
                return true;  // Solution found
            }

            // If placing queen doesn't lead to a solution, backtrack
            board[row][col] = 0;
        }
    }

    // If the queen can't be placed in any column, return false
    return false;
}
// Function to solve the N Queens problemvoid NQueens(int N) {
    int board[MAX][MAX] = {0};  // Initialize the chessboard with all 0s

    // Start from the first row
    if (solveNQueens(board, 0, N)) {
        printSolution(board, N);
    } else {
        printf("Solution does not exist for N = %d\n", N);
    }
}
int main() {
    int N;

    // Taking input for the size of the chessboard
    printf("Enter the value of N: ");
    scanf("%d", &N);

    // Solve the N Queens problem
    NQueens(N);

    return 0;
}
```

Explanation of the Code

Input:

1. The program takes the number of queens N as input. The user specifies the size of the chessboard, which will be an N x N grid.

isSafe Function:

1. This function checks if it is safe to place a queen at position (row, col) on the chessboard. The safety check includes:

    1. No queen should be in the same column (board[i][col] for all rows i).
    2. No queen should be in the upper left diagonal.
    3. No queen should be in the upper right diagonal.

solveNQueens Function:

1. This is the backtracking function that tries to place queens row by row.
2. For each row, it tries placing a queen in all columns one by one and checks whether it's safe using the isSafe function.
3. If placing a queen leads to a solution, it recursively attempts to place queens in the subsequent rows.
4. If placing a queen leads to an invalid configuration (i.e., no solution), it backtracks by removing the queen and tries the next column.

printSolution Function:

1. This function prints the chessboard with the queens placed. A 1 represents a queen, and a 0 represents an empty square.

NQueens Function:

1. This function initializes the chessboard and starts the backtracking process from the first row. It prints the solution if one is found.

Main Function:

1. Takes the value of N from the user and calls the NQueens function to solve the problem.

Sample Output

Enter the value of N: 4

```
0 0 1 0
1 0 0 0
0 0 0 1
0 1 0 0
```

This output represents one of the possible solutions for the 4 Queens problem. The queens are placed at the following positions:

- Queen 1 at (0, 2)
- Queen 2 at (1, 0)
- Queen 3 at (2, 3)

- Queen 4 at (3, 1)

Time Complexity

- **Time Complexity**: The backtracking algorithm has a time complexity of **O(N!)** in the worst case, as it may explore all possible arrangements of queens.
- **Space Complexity**: The space complexity is **O(N^2)** due to the recursive stack and the chessboard used to store the positions of the queens.